INFORME FASE 3

Este documento presenta la Etapa 3 conforme a la consigna oficial (págs. 13–14). Los resultados provienen de ejecuciones y controles en PostgreSQL 18.1. La reproducibilidad ya fue verificada mediante una segunda carga small y comparación de las 27 tablas. El procedimiento de carga, sus requisitos y las instrucciones de ejecución se documentan en db/seed/README.md.

1. Objetivo y Alcance

Se desarrolló una carga automatizada de datos para PostgreSQL, con información de catálogos, producción vitivinícola, movimientos logísticos, ventas y auditoría. El objetivo de volumen fijado por la cátedra exige poblar todas las tablas, superar las 10.000 filas en al menos tres de ellas y las 100.000 en al menos una.

Se utilizaron los perfiles small y full. La carga full fue ejecutada sobre una base nueva denominada bodegas_atuel_fase3_full_final, con las siete migraciones aplicadas y con las claves foráneas, restricciones y triggers activos.

2. Catálogos, scripts y orden de carga

Los catálogos de cepas, maridajes, localidades y departamentos de Mendoza y tipos de vino se incorporan desde archivos CSV. Sus fuentes y criterios de selección están documentados en db/seed/README.md; los datos de referencia no se generan mediante Faker. Los datos ficticios de bodegas, fincas, clientes y operaciones se generan con parámetros y generadores pseudoaleatorios.

La implementación se organiza en db/seed/: config.py contiene perfiles y conexiones; run_seed.py coordina la ejecución; control_integridad.sql agrupa consultas de validación; prueba_rendimiento.py compara dos estrategias de inserción; los cuatro módulos de carga están en db/seed/modules/.

2.1 Esquema de dependencias

El orden de los módulos respeta las dependencias de claves foráneas. Las referencias reales se comparten en un diccionario refs para evitar suponer identificadores generados por las secuencias de PostgreSQL.

CATALOGOS → PRODUCCION → LOGISTICA → VENTAS → RN-13

localidad → sala_venta → transferencia → renglon_transferencia
localidad → cliente → comprobante → renglon_comprobante
finca → parcela → lote_uva → analisis_calidad
bodega → vino → vino_cepa
vino → auditoria_precio

En términos de tablas, los catálogos y maestros preceden a fincas, parcelas, lotes y vinos; luego se procesan ingresos y transferencias para generar existencias, por último, se cargan clientes, comprobantes y renglones de venta. Tras ventas se efectúa una actualización de precio para generar la auditoría RN-13.

3. Coherencia de datos y estrategia frente a triggers

La generación utiliza semillas independientes por módulo, de forma que los datos aleatorios de uno no alteran la secuencia de los siguientes. Las cosechas, análisis, lotes y vinos conservan relaciones coherentes; la composición vino-cepa se genera con porcentajes que suman 100 %. Los clientes se asocian a localidades y tipos válidos; los permisos corresponden a clientes exportadores.

Se mantuvieron activos los triggers, las claves foráneas y las restricciones. Los ingresos y las transferencias se procesaron mediante los procedimientos de la base; no se modificaron directamente las existencias para evitar duplicar movimientos. Los descuentos, subtotales y totales son gestionados por los mecanismos de la base, y los renglones vendidos descuentan stock mediante triggers.

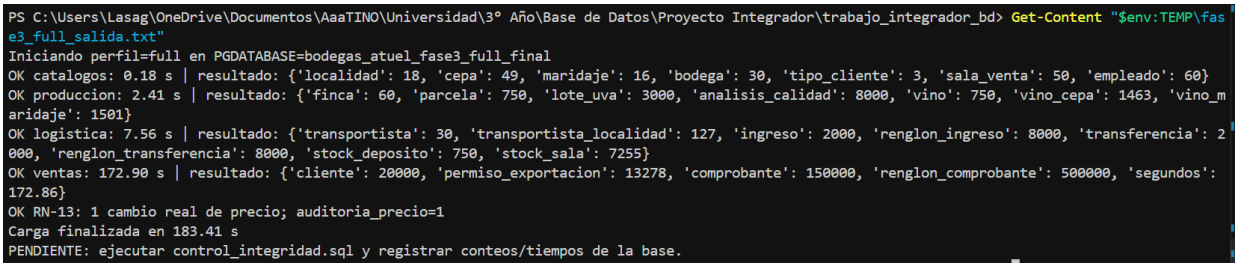
Esta decisión permite comprobar las mismas reglas que usaría el sistema en funcionamiento. Su principal costo es el tiempo de ejecución —sobre todo en ventas, por la cantidad de renglones y actualizaciones relacionadas— y el riesgo de rechazos si faltan existencias. Deshabilitar triggers habría simplificado la carga histórica, pero exigiría reconstruir y validar manualmente sus efectos.

La RN-13 se validó mediante una actualización real sobre un vino: UPDATE vino SET precio = precio + 1.00. El trigger creó una fila en auditoria_precio. La actualización se hace después de registrar las ventas, por lo que no reescribe los precios unitarios ya almacenados en comprobantes.

Respecto del tiempo, las transferencias y ventas se registran en la fecha operativa de PostgreSQL (CURRENT_DATE) debido a las restricciones de la RN-11. Por eso el historial de producción usa fechas previas y la reproducibilidad de fechas operativas se compara dentro del mismo día.

4. Carga full: Volúmenes y Tiempos reales

La siguiente evidencia corresponde a la ejecución de run_seed.py --run en la base final, con PROFILE=full. El programa confirmó la ejecución de catálogos, producción, logística, ventas y la auditoría RN-13; la carga completa tardó 183,41 segundos. El módulo de ventas concentró el mayor tiempo debido a sus 150.000 comprobantes y 500.000 renglones.



Ejecución de la carga full por módulos y confirmación de RN-13

En lugar de duplicar los conteos en una tabla de Word, a continuación, se muestra la salida del control SQL que informa cada tabla y los tres resúmenes de volumen. Se obtuvo un resultado OK en las 27 tablas; cuatro superaron 10.000 filas y dos superaron 100.000.

control	cantidad	estado
analisis_calidad	8000	OK
auditoria_precio	1	OK
bodega	30	OK
cepa	49	OK
cliente	20000	OK
comprobante	150000	OK
empleado	60	OK
finca	60	OK
ingreso	2000	OK
localidad	18	OK
lote_uva	3000	OK
maridaje	16	OK
parcela	750	OK
permiso_exportacion	13278	OK
renglon_comprobante	500000	OK
renglon_ingreso	8000	OK
renglon_transferencia	8000	OK
sala_venta	50	OK
stock_deposito	750	OK
stock_sala	7255	OK
tipo_cliente	3	OK
transferencia	2000	OK
transportista	30	OK
transportista_localidad	127	OK
vino	750	OK
vino_cepa	1463	OK
vino_maridaje	1501	OK
RESUMEN: tablas con al menos una fila	27	OK
RESUMEN: tablas con mas de 10000 filas	4	OK
RESUMEN: tablas con mas de 100000 filas	2	OK

Conteo de las 27 tablas y cumplimiento de los mínimos de volumen

5. Consultas de control e integridad

Se ejecutó db/seed/control_integridad.sql sobre la base full y posteriormente sobre la base restaurada. El archivo comprueba los conteos, las referencias huérfanas, el stock negativo y las diferencias entre importes guardados y recalculados. Las consultas completas quedan disponibles en el repositorio, en vez de copiar su SQL extenso dentro del informe.

5.1. Integridad referencial

La consulta verificó 32 relaciones de claves foráneas y devolvió cero registros huérfanos en todos los casos. La captura siguiente muestra la tabla de relaciones; la salida completa terminó con “(32 rows)”.

relacion	huerfanos	estado
fk_analisis_calidad_lote_uva	0	OK
fk_auditoria_precio_vino	0	OK
fk_cliente_localidad	0	OK
fk_cliente_tipo_cliente	0	OK
fk_comprobante_cliente	0	OK
fk_comprobante_sala_venta	0	OK
fk_empleado_sala_venta	0	OK
fk_ingreso_bodega	0	OK
fk_ingreso_transportista	0	OK
fk_lote_uva_bodega	0	OK
fk_lote_uva_parcela	0	OK
fk_parcela_cepa	0	OK
fk_parcela_finca	0	OK
fk_permiso_exportacion_cliente	0	OK
fk_renglon_comprobante_comprobante	0	OK
fk_renglon_comprobante_vino	0	OK
fk_renglon_ingreso_ingreso	0	OK
fk_renglon_ingreso_vino	0	OK
fk_renglon_transferencia_transferencia	0	OK
fk_renglon_transferencia_vino	0	OK
fk_sala_venta_localidad	0	OK
fk_stock_deposito_vino	0	OK
fk_stock_sala_sala_venta	0	OK
fk_stock_sala_vino	0	OK
fk_transferencia_sala_venta	0	OK

```
fk_vino_cepa_cepa                    |         0 | OK
fk_vino_cepa_vino                    |         0 | OK
fk_vino_maridaje_maridaje            |         0 | OK
fk_vino_maridaje_vino                |         0 | OK
(32 rows)
```

Control de integridad referencial de las claves foráneas

5.2. Existencias e importes comerciales

Los controles de stock_deposito y stock_sala devolvieron cero filas con cantidades negativas. La consulta que compara subtotal, descuento y total de los comprobantes con los importes esperados no devolvió registros (0 rows).

```
     tabla      | filas_negativas | estado
----------------+-----------------+--------
 stock_deposito |               0 | OK
 stock_sala     |               0 | OK
(2 rows)

 nro_comprobante | subtotal_guardado | subtotal_esperado | descuento_guardado | descuento_esperado | total_guardado | total_esperado | total_renglon
es_guardados
-----------------+-------------------+-------------------+--------------------+--------------------+----------------+----------------+-------------
-------------
(0 rows)
```

Controles de stock negativo y comprobantes con importes inconsistentes

5.3. Conservación de stock por sala y vino

Además, se realizó una conciliación independiente por cada combinación de sala y vino, considerando las transferencias procesadas, los renglones vendidos y el stock final. Se verificó la igualdad 89.537.974 - 999.954 = 88.538.020, con cero diferencias en el conjunto de combinaciones

```
 botellas_transferidas | botellas_vendidas | stock_actual | diferencias_stock
-----------------------+-------------------+--------------+-------------------
              89537974 |            999954 |     88538020 |                 0
(1 row)
```

Conciliación de transferencias, ventas y existencias finales

En la validación específica de ventas también se comprobaron documentos de cliente no duplicados, permisos válidos de exportación, descuentos correctos, ausencia de vinos repetidos por comprobante y ventas respaldadas por transferencias previas compatibles; no se informaron inconsistencias en estas consultas.

6. Comparación de rendimientos de inserción

La cátedra solicita comparar inserciones individuales con un método de carga masiva. Para la prueba principal se usó una tabla temporal basada en cliente (sin copiar FK ni triggers), con los mismos 10.000 registros en cada método. Se midieron tres repeticiones por INSERT fila a fila y COPY FROM STDIN; el cronómetro incluyó inserción y COMMIT, pero no preparación, limpieza ni COUNT.

```
Base: bodegas_atuel_rn13_small | PostgreSQL: 18.1
Muestra identica: 10000 filas | repeticiones: 3
Tabla temporal basada en cliente; sin FK ni triggers
Cronometro: insercion + COMMIT; limpieza y COUNT excluidos
Repeticion 1: fila_por_fila=3.399 s; COPY=0.103 s
Repeticion 2: fila_por_fila=3.510 s; COPY=0.134 s
Repeticion 3: fila_por_fila=3.501 s; COPY=0.148 s

RESULTADO (medianas)
Fila por fila: 3.501 s (10000 INSERT)
COPY:          0.134 s (1 operacion COPY)
Aceleracion COPY: 26.14x
Ahorro de tiempo: 96.2%
Verificacion: misma cantidad de filas en cada repeticion
Seguridad: no se modificaron tablas permanentes
Evidencia: C:\Users\Lasag\AppData\Local\Temp\prueba_rendimiento.csv
```

Mediciones de tres repeticiones: INSERT individuales frente a COPY

La mediana de INSERT individual fue 3,501 s, frente a 0,134 s con COPY. La carga mediante COPY resultó 26,14 veces más rápida, equivalente a una reducción del 96,2 % del tiempo observado. También se realizó una prueba separada con INSERT multi-VALUES en lotes de 1.000 registros: su mediana fue 0,448 s frente a 3,461 s para inserciones individuales. Los factores de ambas pruebas no deben confundirse con la duración de la carga full.

La diferencia observada se explica principalmente por la menor cantidad de operaciones y viajes de consulta al servidor. El ensayo aísla el mecanismo de inserción; los triggers y las restricciones utilizadas durante la carga real implican un costo adicional que no está incluido en esos tiempos.

7. Reproducibilidad con semilla fija

Se compararon dos ejecuciones independientes con PROFILE=small, SEED=20261027 y BATCH_SIZE=1000, realizadas en bodegas_atuel_rn13_small y bodegas_atuel_repro_small. La segunda terminó correctamente (código de salida 0).

Para ir más allá de los conteos, se calcularon huellas SHA-256 a partir de representaciones JSON de las filas ordenadas de cada tabla. Las 27 tablas tuvieron huellas coincidentes y la comparación final indicó cero tablas diferentes. Se excluyó únicamente auditoria_precio.fecha_hora, porque refleja el momento real de ejecución; las fechas dependientes de CURRENT_DATE coincidieron al realizar las pruebas el mismo día.

```
COMPARACION DE REPRODUCIBILIDAD
Semilla: 20261027 | Perfil: small
Auditoria: se excluye fecha_hora
------------------------------------------------------------
analisis_calidad                 50 /    50 IGUAL
auditoria_precio                  1 /     1 IGUAL
bodega                            5 /     5 IGUAL
cepa                             12 /    12 IGUAL
cliente                         100 /   100 IGUAL
comprobante                     100 /   100 IGUAL
empleado                          5 /     5 IGUAL
finca                             5 /     5 IGUAL
ingreso                          10 /    10 IGUAL
localidad                        10 /    10 IGUAL
lote_uva                         50 /    50 IGUAL
maridaje                         10 /    10 IGUAL
parcela                          25 /    25 IGUAL
permiso_exportacion              57 /    57 IGUAL
renglon_comprobante             250 /   250 IGUAL
renglon_ingreso                  20 /    20 IGUAL
renglon_transferencia            20 /    20 IGUAL
sala_venta                        5 /     5 IGUAL
stock_deposito                   15 /    15 IGUAL
stock_sala                       19 /    19 IGUAL
tipo_cliente                      3 /     3 IGUAL
transferencia                    10 /    10 IGUAL
transportista                     5 /     5 IGUAL
transportista_localidad          19 /    19 IGUAL
vino                             25 /    25 IGUAL
vino_cepa                        53 /    53 IGUAL
vino_maridaje                    47 /    47 IGUAL
------------------------------------------------------------
Tablas comparadas: 27
Tablas con diferencias: 0
RESULTADO: REPRODUCIBILIDAD COMPROBADA
```

Comparación reproducible de las tablas con la misma semilla

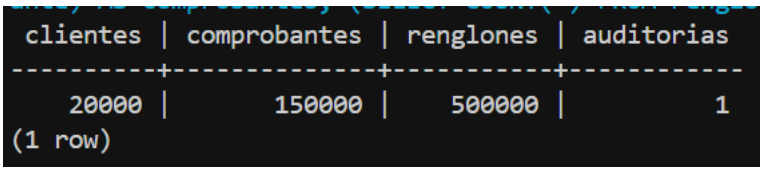
El cierre de la misma prueba indicó: «Tablas comparadas: 27», «Tablas con diferencias: 0» y «RESULTADO: REPRODUCIBILIDAD COMPROBADA». Este resultado avala la reproducibilidad de los datos deterministas de small bajo las condiciones documentadas; no implica igualdad byte a byte de los campos tomados del reloj en ejecuciones de días distintos.

8. Respaldo y recuperación

Se generó un volcado completo con pg_dump y se comprimió como dump_completo.sql.gz. El archivo tiene 10.502.108 bytes comprimidos y 31.002.722 bytes al descomprimir; la lectura completa del GZIP terminó con código 0. Se calculó la huella SHA-256 para identificar la copia utilizada:

D1A066940B9429866FB490D7CE5675159218BDB7F7376CB9582C490696DD0B78

El volcado se restauró en una base descartable nueva denominada bodegas_atuel_restauracion. Allí se comprobó la recuperación de 20.000 clientes, 150.000 comprobantes, 500.000 renglones y una fila de auditoría. La ejecución posterior de control_integridad.sql volvió a mostrar las 27 tablas pobladas, sin filas huérfanas, sin stock negativo ni comprobantes inconsistentes.



```
 clientes | comprobantes | renglones | auditorias
----------+--------------+-----------+------------
    20000 |       150000 |    500000 |          1
(1 row)
```

Conteos recuperados desde la base restaurada

9. Conclusión técnica

La Fase 3 alcanzó los criterios de volumen y consistencia verificados: 27 tablas pobladas, cuatro tablas con más de 10.000 registros, dos con más de 100.000, 500.000 renglones de venta, 32 controles de FK sin huérfanos, cero existencias negativas y cero inconsistencias de importes. La auditoría RN-13 contiene un cambio efectivo de precio, la conciliación de stock cierra sin diferencias y el dump comprimido fue restaurado correctamente.

La reproducibilidad se comprobó entre dos ejecuciones small con la misma semilla y 27 tablas coincidentes, dejando documentada la excepción del timestamp de auditoría y la dependencia de CURRENT_DATE.

10. Fuentes y ubicación de evidencias

Consigna: TPI Bodegas del Atuel, sección «Etapa 3. Carga masiva de datos», págs. 13–14. Código: repositorio github.com/videlalauti/trabajo_integrador_bd, rama main. Archivos relevantes: db/seed/README.md, config.py, run_seed.py, control_integridad.sql, modules/*.py y prueba_rendimiento.py.

Evidencias del equipo: fase3_full_salida.txt, fase3_full_integridad.txt, fase3_full_stock.txt, prueba_rendimiento.csv, prueba_rendimiento_salida.txt, reproducibilidad_fase3.txt, repro_small_salida.txt, dump_completo.sql.gz y salida de restauración. Los archivos de evidencia y el dump se conservan fuera del repositorio hasta definir el paquete final de entrega.